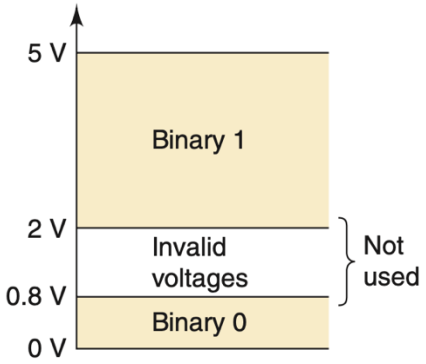
How Do Computers Work?

Pei Zou

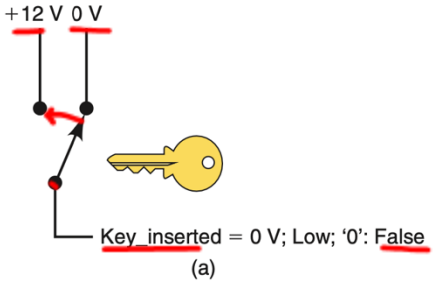
Digital Systems, High/Low Voltage

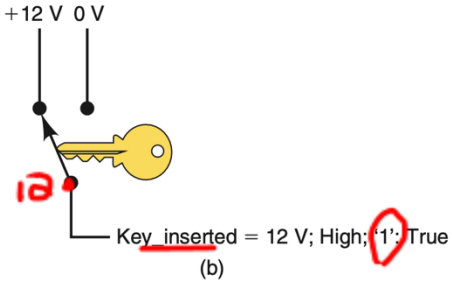
- Digital systems deal with two distinct digits: 0 and 1, called “binary digits”, or “bit”;
- 0 and 1 can represent many different things. Most significant among them:
  - Low voltage → 0; high voltage → 1;
  - Distinct, “physical” states: Open / Closed; CW / CCW; Off / On;
  - Quantities;
  - Information (encoding/decoding).

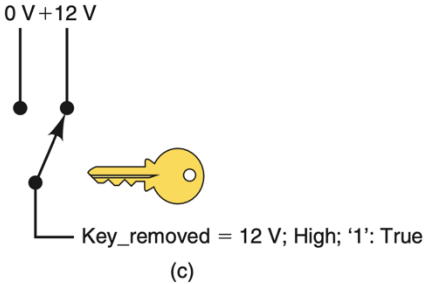


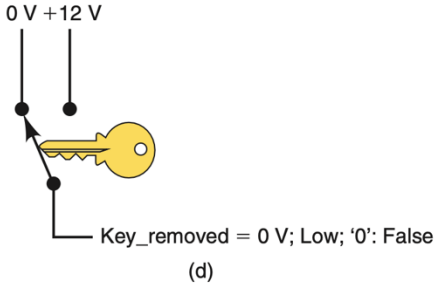
When we talk about 0 and 1, in the physical “chip and wire” circuits, it’s high voltage and low voltage.

Distinct Physical States









0V when key is removed, 12V when key is inserted:

- Key not inserted: 0V⇔0⇔ key_inserted False;
- Key is inserted: 12V⇔1⇔ key_inserted True;
- (key_inserted) ACTIVE HIGH;
- (key_removed) ACTIVE LOW.

12V when key is removed, 0V when key is inserted:

- Key is removed: 12V⇔1⇔ key_removed True;
- Key is not removed: 0V⇔1⇔ key_removed False;
- (key_removed) ACTIVE HIGH;
- (key_inserted) ACTIVE LOW.

While high voltage = 1, low voltage = 0 is almost always true, associating ”logic state” to digits depends on the definition of the “logic state” and on wiring.

Quantity: Decimal

How do we humans count:

- 10 distinct numerals, symbols, digits or human fingers: 0, 1, …, 9;
- Cout from 0 to 9 with the 10 distinct numerals;
- Counting 1 more from 9, when the 10 distinct numerals are exhausted, we use 10, where the *position* (the "tens" position) of the numeral 1 means that it represents 10;
- Then we increment the "unit" position and count 10, 11, …, 19;
- When the unit position exhausts the 10 distinct numerals, we increment 1 in the tens position and count 20, 21, …, 29.

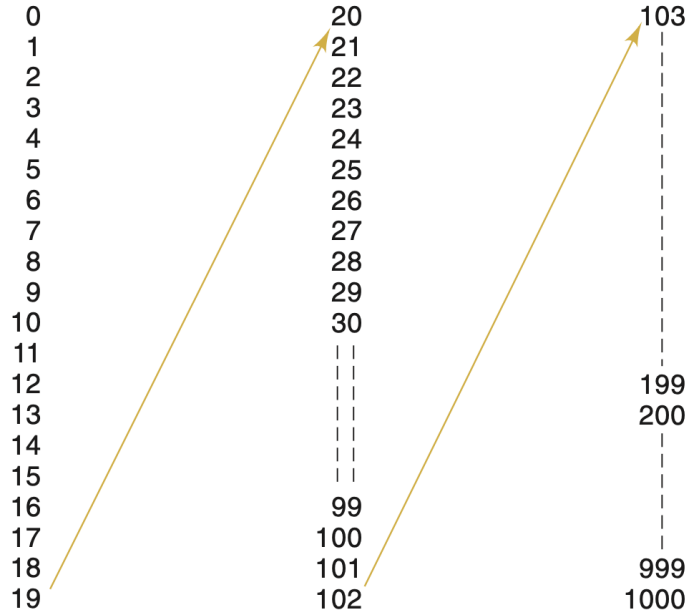
The value of each numeral in a number is determined by the numeral itself and its position:

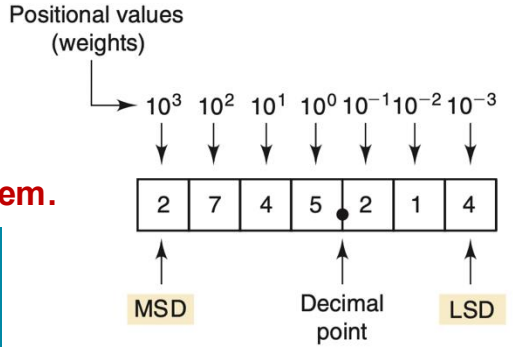
$$2745.214$$
$$= 2 \times 10^3 + 7 \times 10^2 + 4 \times 10^1 + 5 \times 10^0 + 2 \times 10^{-1} + 1 \times 10^{-2} + 4 \times 10^{-3}$$

With 2 digits, can count 100 (= 10^2) numbers from 0 to 99 (= $10^2 - 1$):

- With N digits, can count 10^N numbers from 0 to $10^N - 1$.

This 10-based counting, quantity-representing system is called the decimal system.





Quantity: Binary

How do computing machines count:

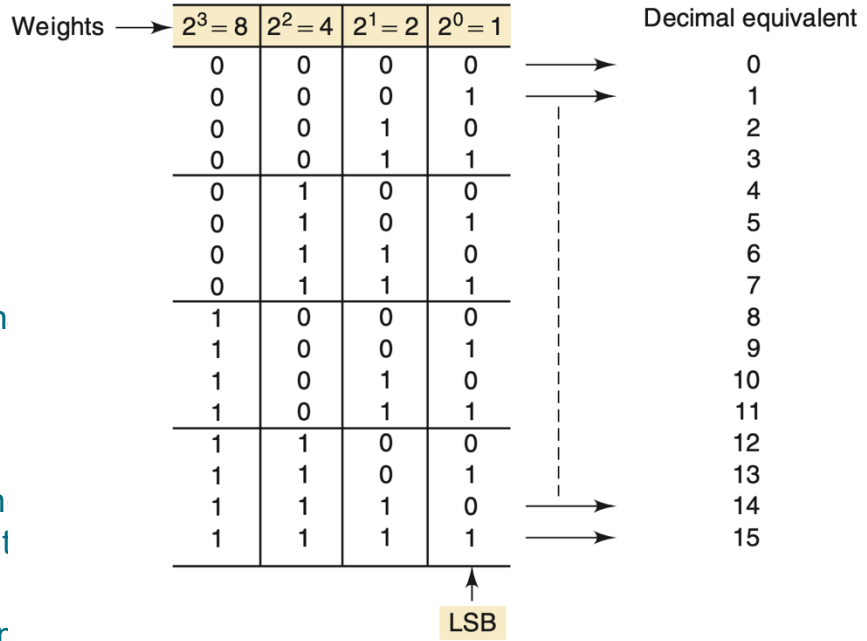
- 2 distinct numerals, symbols, digits or machine states: 0, 1;
- Cout from 0 to 1 with the 2 distinct numerals;
- Counting 1 more from 1, when the 2 distinct numerals are exh we use 10, where the *position* of the numeral 1 means that it represents 2;
- Then we increment the “unit” position and count 10, 11;
- When the unit position exhausts the 2 distinct numerals, we in 1 in the 2nd position and find that the 2nd position has exhaust numerals, so we use the 3rd position and count 100.

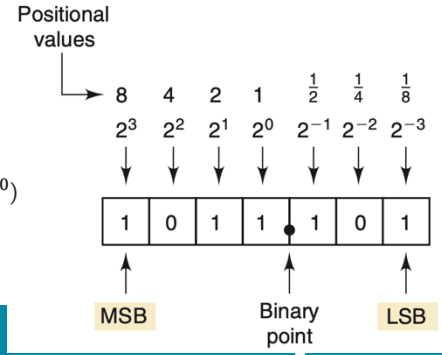
The value of each numeral in a number is determined by the nur itself and its position:

With 2 digits, can count 4 (= 2^2) numbers from 0 to 11 (= $2^2 - 1 = 3$):

- With N digits, can count 2^N numbers from 0 to $2^N - 1$.

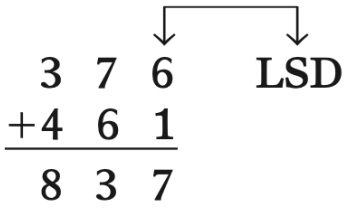
This 2-based counting, quantity-representing system is called the binary system.





$$1011.101_2 = (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) + (1 \times 2^{-1}) + (0 \times 2^{-2}) + (1 \times 2^{-3})$$
$$= 8 + 0 + 2 + 1 + 0.5 + 0 + 0.125$$
$$= 11.625_{10}$$

Binary Addition, Subtraction



- Decimal addition and subtraction follow the “long-form” rules;
- Binary numbers are counted exactly the same way as decimal numbers, except with only two available digits;
- Therefore their additions and subtractions are done exactly like decimals, except with only two available digits and thus a lot less number of ”scenarios”;
- Computers/digital circuits don’t calculate this way. We’ll see how they do them.

$$0 + 0 = 0$$
$$1 + 0 = 1$$
$$1 + 1 = 10 = 0 + \text{carry of 1 into next position}$$
$$1 + 1 + 1 = 11 = 1 + \text{carry of 1 into next position}$$

$$0 - 0 = 0$$
$$1 - 1 = 0$$
$$1 - 0 = 1$$
$$0 - 1 \Rightarrow \text{needs to borrow} \Rightarrow 10 - 1 = 1$$

$$\begin{array}{r} 011\ (3) \\ +\ 110\ (6) \\ \hline 1001\ (9) \end{array} \qquad \begin{array}{r} 1001\ (9) \\ +\ 1111\ (15) \\ \hline 11000\ (24) \end{array} \qquad \begin{array}{r} 11.011\ (3.375) \\ +\ 10.110\ (2.750) \\ \hline 110.001\ (6.125) \end{array}$$

$$\begin{array}{r} 110\ (6) \\ -\ 010\ (2) \\ \hline 100\ (4) \end{array} \qquad \begin{array}{r} 11011\ (27) \\ -\ 01101\ (13) \\ \hline 1110\ (14) \end{array} \qquad \begin{array}{r} 1000.10\ (8.50) \\ -\ 0011.01\ (3.25) \\ \hline 101.01\ (5.25) \end{array}$$

ASCII Code

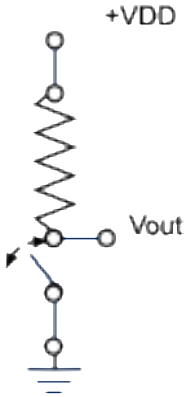
	0	1	2	3	4	5	6	7
0	NUL	DLE	space	0	@	P	`	p
1	SOH	DC1 XON	!	1	A	Q	a	q
2	STX	DC2	"	2	B	R	b	r
3	ETX	DC3 XOFF	#	3	C	S	c	s
4	EOT	DC4	$	4	D	T	d	t
5	ENQ	NAK	%	5	E	U	e	u
6	ACK	SYN	&	6	F	V	f	v
7	BEL	ETB	'	7	G	W	g	w
8	BS	CAN	(	8	H	X	h	x
9	HT	EM	)	9	I	Y	i	y
A	LF	SUB	*	:	J	Z	j	z
B	VT	ESC	+	;	K	[	k	{
C	FF	FS	,	<	L	\	l	|
D	CR	GS	-	=	M	]	m	}
E	SO	RS	.	>	N	^	n	~
F	SI	US	/	?	O	_	o	del

TABLE 2-5 Standard ASCII codes.

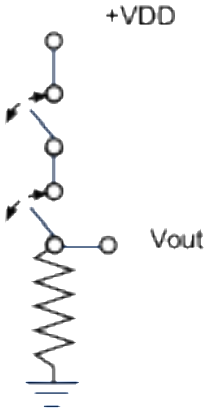
Character	Hex	Decimal	Character	Hex	Decimal	Character	Hex	Decimal	Character	Hex	Decimal
NUL (null)	0	0	Space	20	32	@	40	64	.	60	96
Start Heading	1	1	!	21	33	A	41	65	a	61	97
Start Text	2	2	"	22	34	B	42	66	b	62	98
End Text	3	3	#	23	35	C	43	67	c	63	99
End Transmit.	4	4	$	24	36	D	44	68	d	64	100
Enquiry	5	5	%	25	37	E	45	69	e	65	101
Acknowlege	6	6	&	26	38	F	46	70	f	66	102
Bell	7	7	`	27	39	G	47	71	g	67	103
Backspace	8	8	(	28	40	H	48	72	h	68	104
Horiz. Tab	9	9	)	29	41	I	49	73	i	69	105
Line Feed	A	10	*	2A	42	J	4A	74	j	6A	106
Vert. Tab	B	11	+	2B	43	K	4B	75	k	6B	107
Form Feed	C	12	,	2C	44	L	4C	76	l	6C	108
Carriage Return	D	13	-	2D	45	M	4D	77	m	6D	109
Shift Out	E	14	.	2E	46	N	4E	78	n	6E	110
Shift In	F	15	/	2F	47	O	4F	79	o	6F	111
Data Link Esc	10	16	0	30	48	P	50	80	p	70	112
Direct Control 1	11	17	1	31	49	Q	51	81	q	71	113
Direct Control 2	12	18	2	32	50	R	52	82	r	72	114
Direct Control 3	13	19	3	33	51	S	53	83	s	73	115
Direct Control 4	14	20	4	34	52	T	54	84	t	74	116
Negative ACK	15	21	5	35	53	U	55	85	u	75	117
Synch Idle	16	22	6	36	54	V	56	86	v	76	118
End Trans Block	17	23	7	37	55	W	57	87	w	77	119
Cancel	18	24	8	38	56	X	58	88	x	78	120
End of Medium	19	25	9	39	57	Y	59	89	y	79	121
Substitue	1A	26	:	3A	58	Z	5A	90	z	7A	122
Escape	1B	27	;	3B	59	[	5B	91	{	7B	123
Form Separator	1C	28	<	3C	60	\	5C	92	|	7C	124
Group Separator	1D	29	=	3D	61	]	5D	93	}	7D	125
Record Separator	1E	30	>	3E	62	^	5E	94	~	7E	126
Unit Separator	1F	31	?	3F	63	_	5F	95	Delete	7F	127

“Switch” Logic

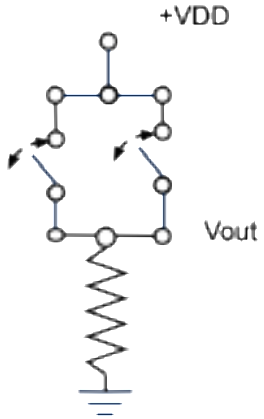
- If we define the state of the switch:
  - Open = 0; close = 1.
- Then we have:
  - NOT, AND, OR logic relationships.
- We can mathematically prove that any logic relationship can be built with these three fundamental building blocks;
- However, the problem with (flip) switches:
  - It’s not “electronic” (inputs not high/low voltages or 1s/0s);
  - We can’t scale;
  - Way too slow.



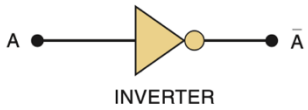
Switch	V_{out}
Open (0)	1 (V_{DD})
Close (1)	0

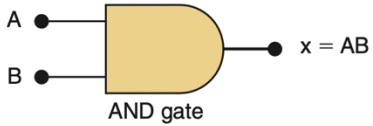


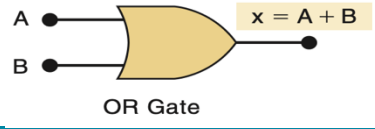
Top	Bottom	V_{out}
Open (0)	Open(0)	0
Open(0)	Close (1)	0
Close (1)	Open (0)	0
Close (1)	Close (1)	1 (V_{DD})



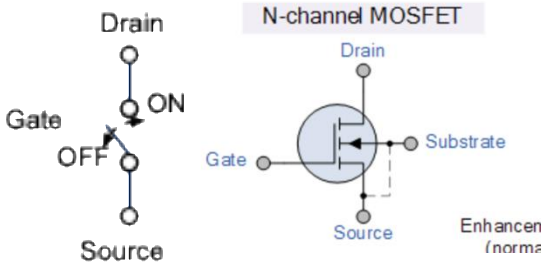
Left	Right	V_{out}
Open (0)	Open (0)	0
Open (0)	Close (1)	1
Close (1)	Open (0)	1
Close (1)	Close (1)	1





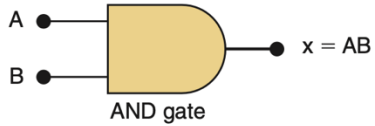


Digital Logic

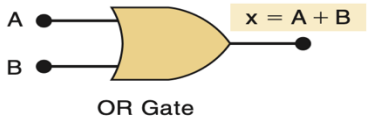


V_{Gate}	Switch
1	Close
0	Open

$V_{gate\text{-}top}$	$V_{gate\text{-}bottom}$	V_{out}
0	0	0
0	1	0
1	0	0
1	1	1

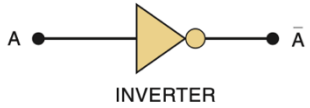


$V_{gate\text{-}left}$	$V_{gate\text{-}r1ght}$	V_{out}
0	0	0
0	1	1
1	0	1
1	1	1



- We use a “magic device” called a transistor to make a voltage controlled switch;
- Then we have complete electronic ways to implement NOT, AND and OR;
- We can construct electronic circuits that implement any “rule” to turn one set of 0s and 1s into another;
- What if we make the rule something meaningful?

V_{Gate}	Switch	V_{out}
0	Open	1 (V_{DD})
1	Close	0

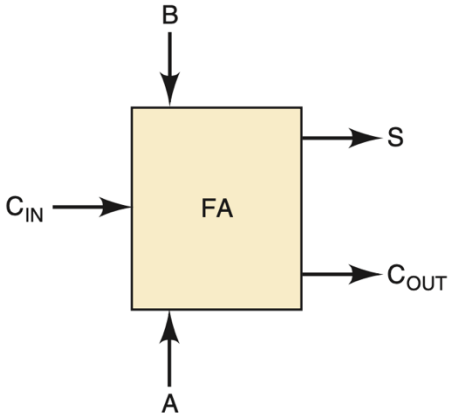


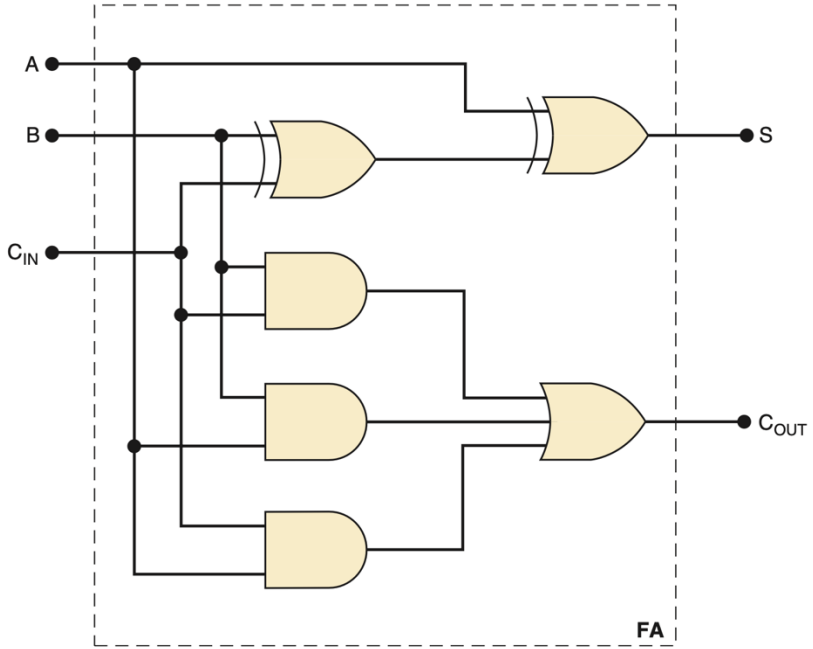
1-Bit Full Adder (FA)

Augend bit input	Addend bit input	Carry bit input		Sum bit output	Carry bit output
A	B	C_{IN}		S	C_{OUT}
0	0	0		0	0
0	0	1		1	0
0	1	0		1	0
0	1	1		0	1
1	0	0		1	0
1	0	1		0	1
1	1	0		0	1
1	1	1		1	1

$$S = A \oplus [B \oplus C_{IN}]$$

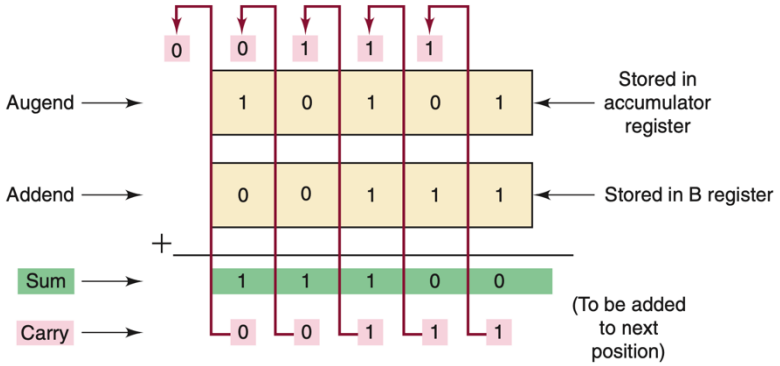
$$C_{OUT} = BC_{IN} + AC_{IN} + AB$$

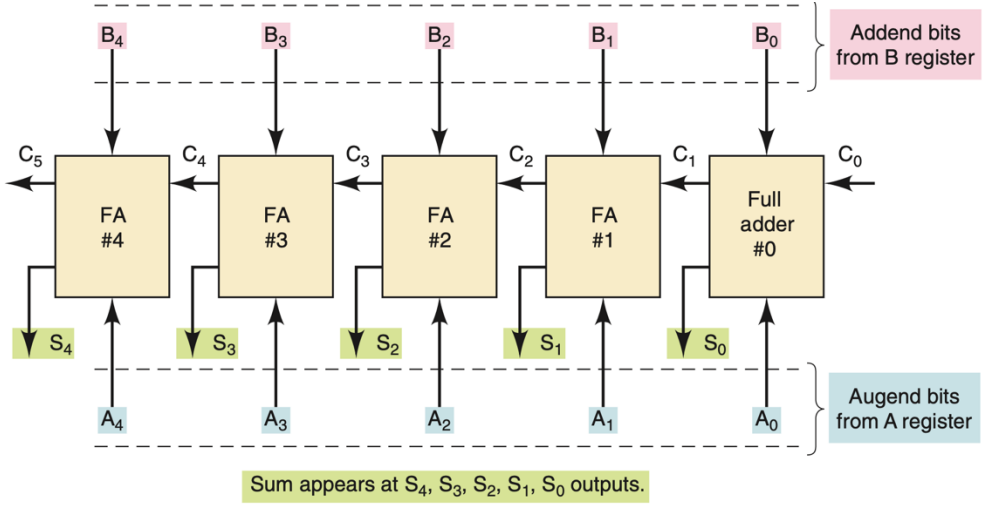




- We can construct electronic circuits that implement any “rule” to turn one set of 0s and 1s into another;
- If we make such rule the same as binary add, then output of the circuit will be result of binary addition;
- “Dumb electronics arranged and connected in such a way that its output happens to be the sum of inputs”.

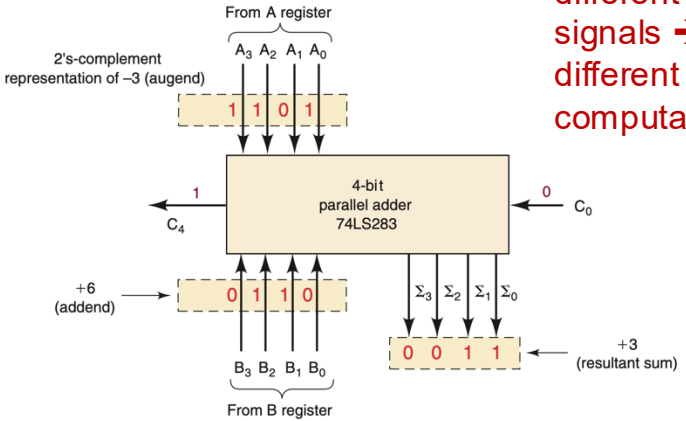
4-Bit Adder Architecture



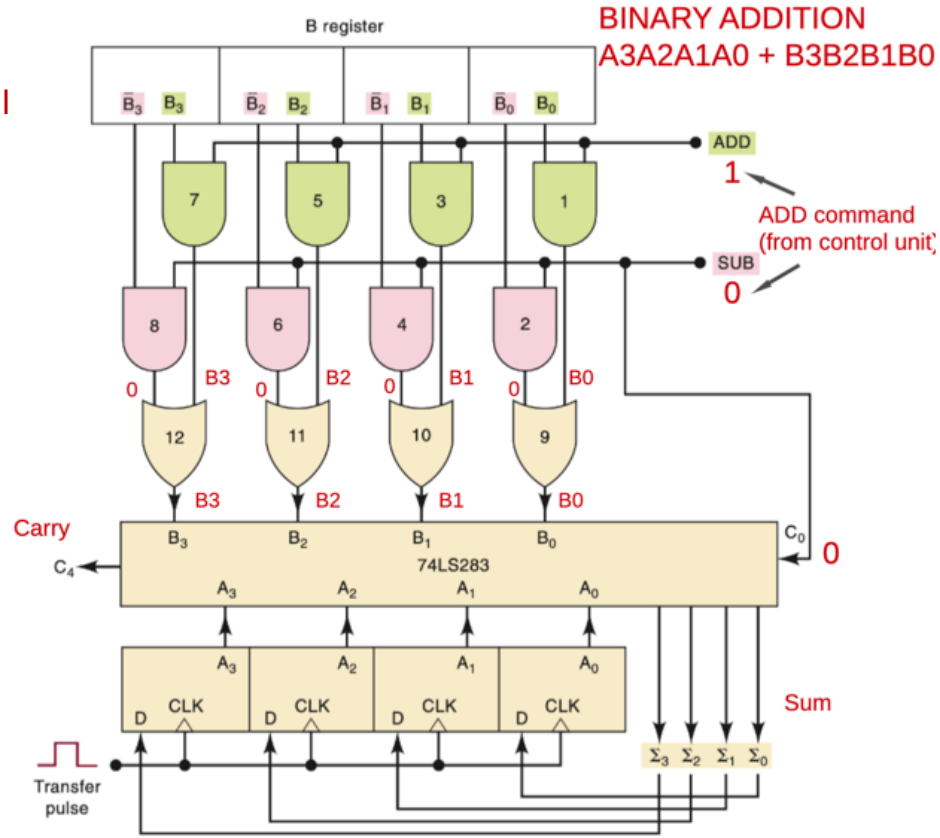


- More bits can put together

4-Bit Add



Same circuitry, different control signals ➔ different computation



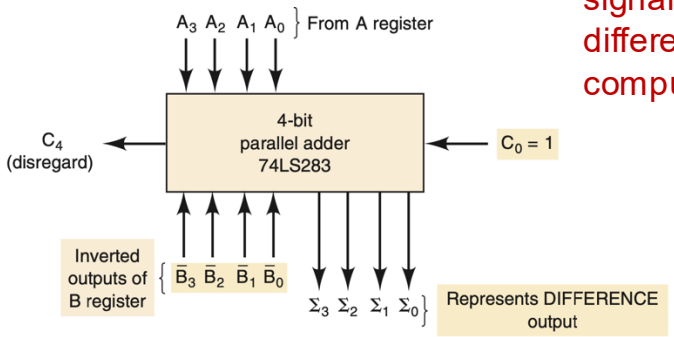
ADD = 1, SUB = 0 control signal: ADD

- B3B2B1B0 shows up at B bus;
- A3A2A1A0 shows up at A bus;
- Sum(B3B2B1B0, A3A2A1A0) → A3A2A1A0;

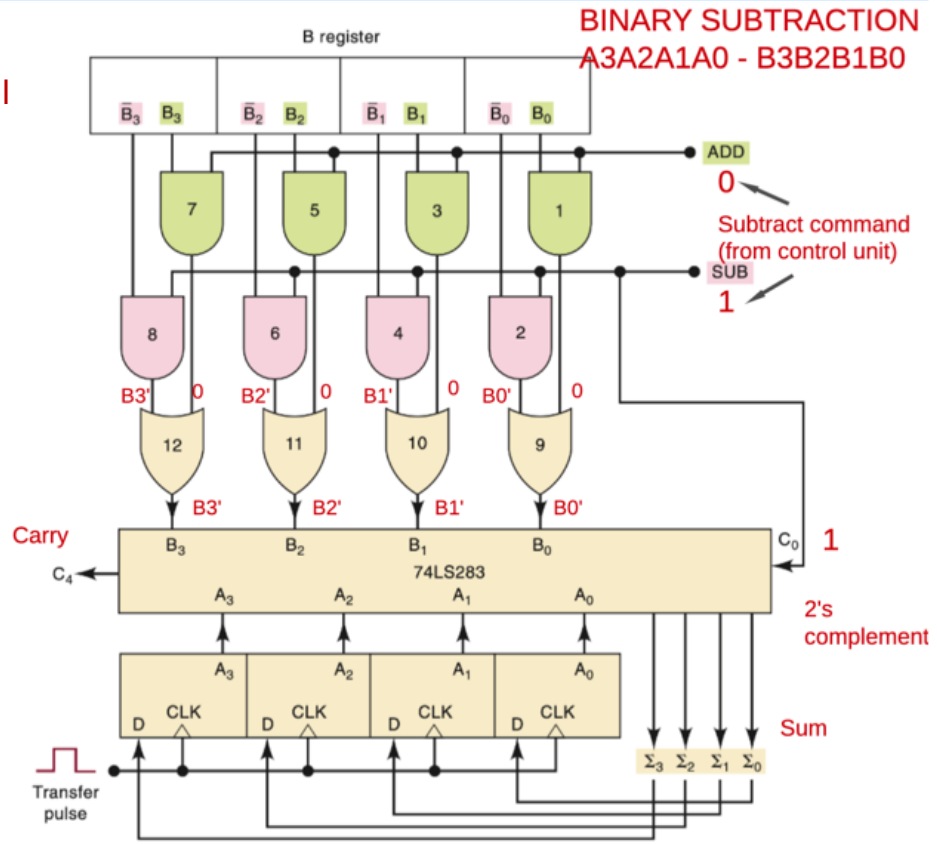
(ADD, SUB) = (1, 0):

- A3A2A1A0 = B3B2B1B0 + A3A2A1A0

4-Bit Subtract



Same circuitry, different control signals ➔ different computation



ADD = 0, SUB = 1 control signal: SUBTRACT

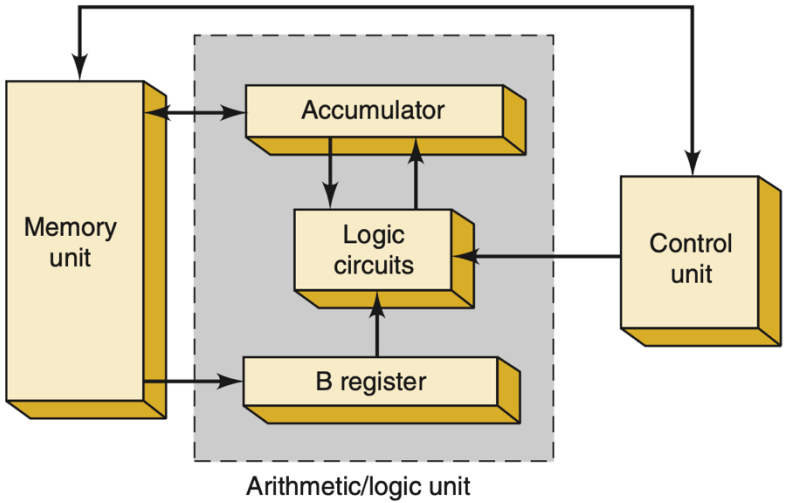
- B3'B2'B1'B0' shows up at B bus, which combined with C_0 = 1, make 2's complement as addend;
- A3A2A1A0 shows up at A bus;
- A3A2A1A0 - B3B2B1B0 → A3A2A1A0.

(ADD, SUB) = (0, 1):

- A3A2A1A0 = A3A2A1A0 - B3B2B1B0.

Digital Computation

- We can construct electronic circuits that implement any "rule" to turn one set of 0s and 1s into another:
  - If we make a circuit implementing the rule of binary arithmetic, then it does arithmetic computation;
  - If we make a circuit implementing encoding/decoding rules, then it encodes/decodes information, making information processing possible;
  - If we make a circuit implementing more complex rules, then it performs more complex tasks.
- "Dumb electronics arranged and connected in such a way that its output happens to be what we want";
- We can make use of the same circuitry to do different things by values of another set of inputs called the control inputs;
- We have a arithmetic/logic unit, or ALU.



More Complex Tasks

If we make a circuit implementing more complex rules, then it performs more complex tasks:

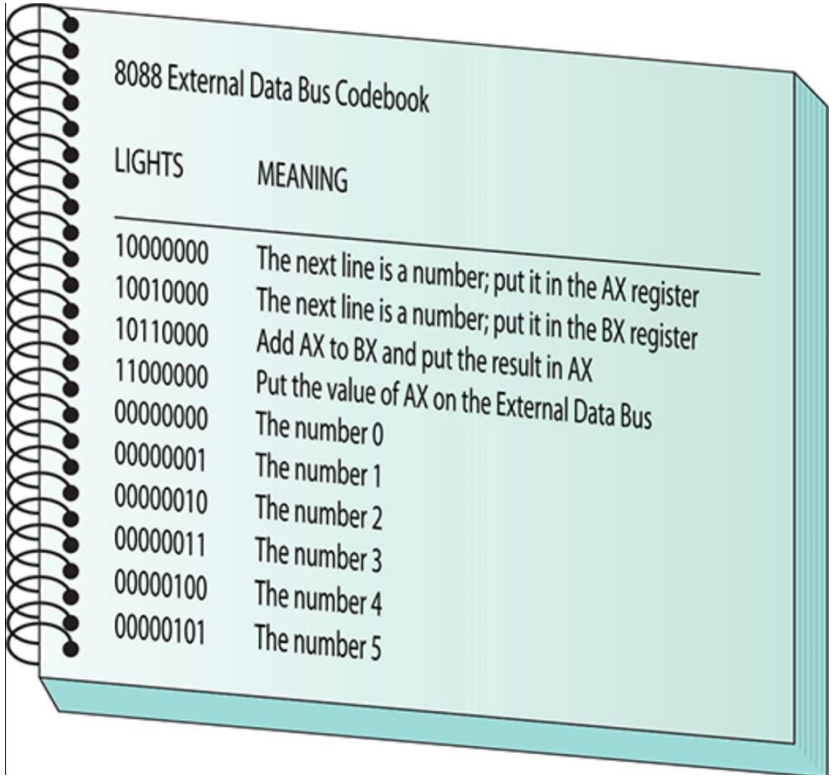
- Add/subtract, encoding/decoding/processing, other operations on 8/16/32/64-bit inputs;
- More complex control tasks with 8/16/32/64-bit control signals;
- Former is data, latter is commands.

“Dumb electronics arranged and connected in such a way that its output happens to be what we want”:

- We can enough of these electronics to perform basic “building-block” computation, into which more complex tasks can be broken.

This is how modern-day digital computation works.

But so far, we only have circuitry that performs tasks by producing outputs *when and if* inputs and commands show up. It *does not* work on its own.



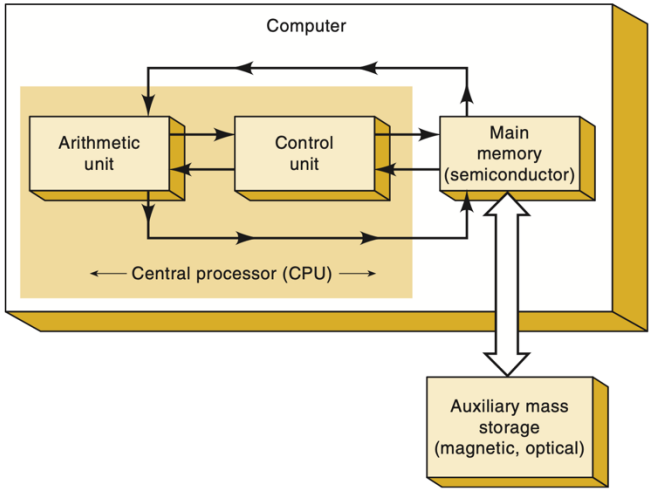
Von Neumann Machine

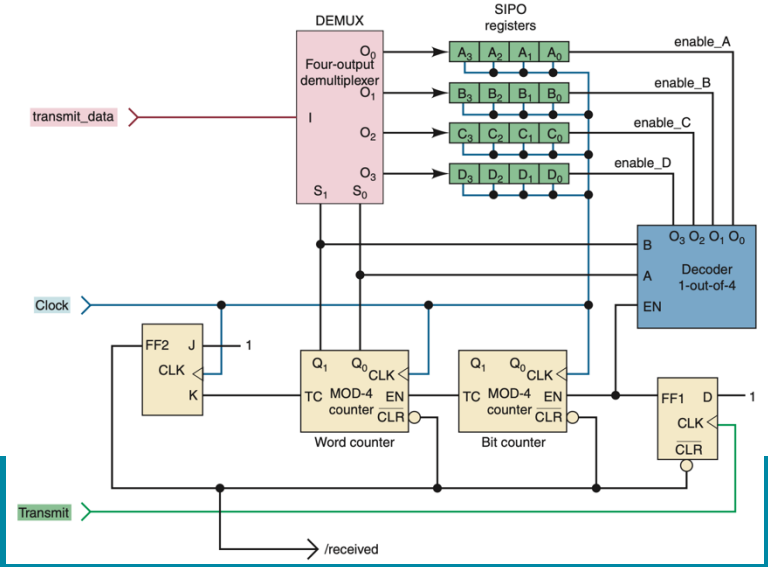
Memory is nothing but a device with two states representing 0 and 1:

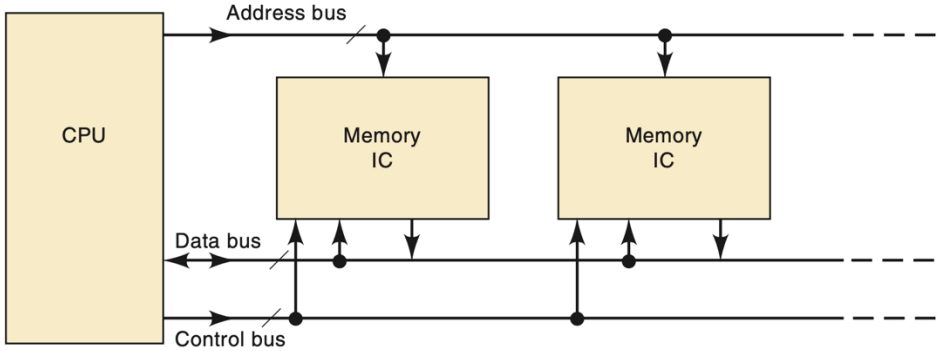
- Charged/discharged in RAM and SSD;
- Magnet changing or not changing directions of its pole in HDD;
- Crystaline/amorphous of material in optical disks.

Use memory to store commands and data;

Use clock to drive action.







From 8088 to Modern CPU

Intel 8088 (1979)

```
ADD AL, immediate8:     00000100 + 8-bit data
MOV AX, BX:             10001001 11011000
JMP address:            11101001 + 16-bit offset
PUSH AX:                01010000
POP BX:                 01011011
CMP AL, immediate8:     00111100 + 8-bit data
```

Modern x86-64 CPU (Intel/AMD)

```
ADD RAX, RBX:           01001000 10000001 11000011  (REX.W + opcode + ModR/M)
VMULPD ymm0, ymm1, ymm2: 11000101 11110101 01011001 11010000  (VEX prefix + opcode
+ ModR/M)
AESENC xmm1, xmm2:      01100110 00001111 00111000 11011100 11001010  (multi-byte
opcode)
MOV RAX, [RBX+RCX*8]:   01001000 10001011 00000100 11001011  (REX + opcode + SIB
byte)
```

Modern GPU (NVIDIA PTX - intermediate form, then compiled to native)

PTX (what you write):

```
add.f32 %f1, %f2, %f3;      // Add two 32-bit floats
ld.global.f32 %f1, [%rd0];  // Load from global memory
mul.wide.u32 %rd1, %r1, 4;  // Multiply and widen to 64-bit
setp.gt.f32 %p1, %f1, %f2;  // Set predicate if f1 > f2
```

Native GPU machine code (proprietary binary, example representation):

```
FFMA R0, R1, R2, R3:    [binary encoding ~64-128 bits, undocumented]
LDG R0, [R1]:           [binary encoding, undocumented]
ISETP.GT P0, R1, R2:    [binary encoding, undocumented]
```

Today's modern CPU:

- Multiple cores, each containing multiple ALUs, run independently;
- More complex instruction sets.

GPU:

- Over 100 cores, within each 100's over "mini-cores" processing in parallel.

Even More Complex Tasks

Building more cores with more complex instruction sets are part of hardware implementation:

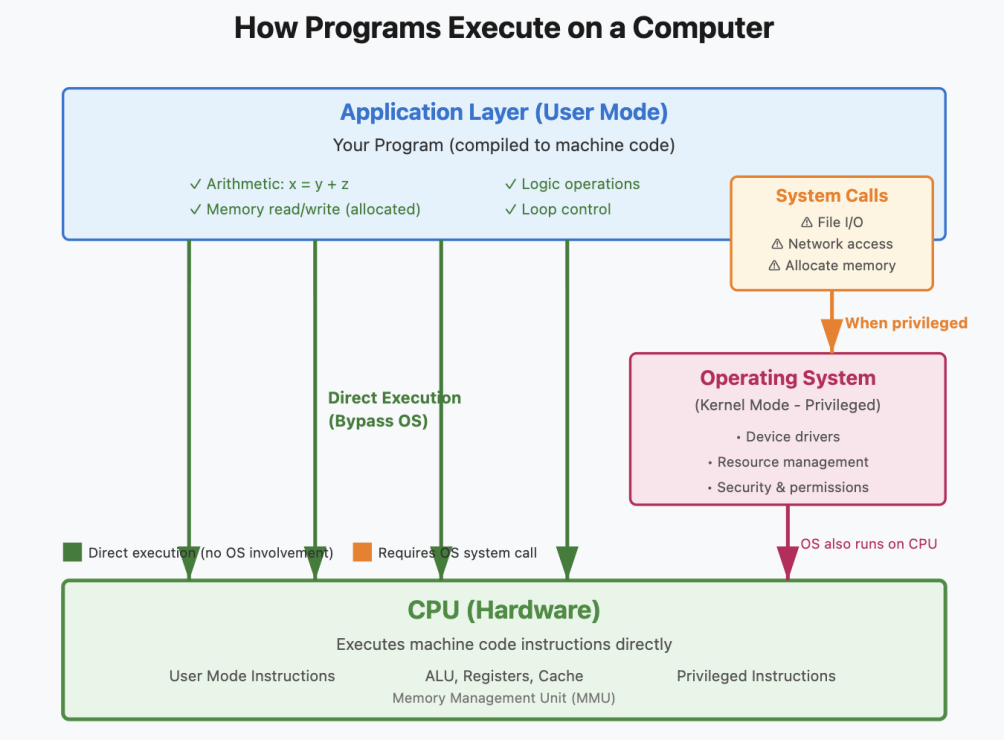
- They still can only handle “elementary” operations.

More complex “human-level” tasks are done through software:

- Software commands through compilation and interpretation eventually convert to “machine codes” of 0s and 1s;
- Machine code is a breakdown of the command into a series of instruction set-level instructions.

Operating system is privileged software:

- Manages devices, resources, security.



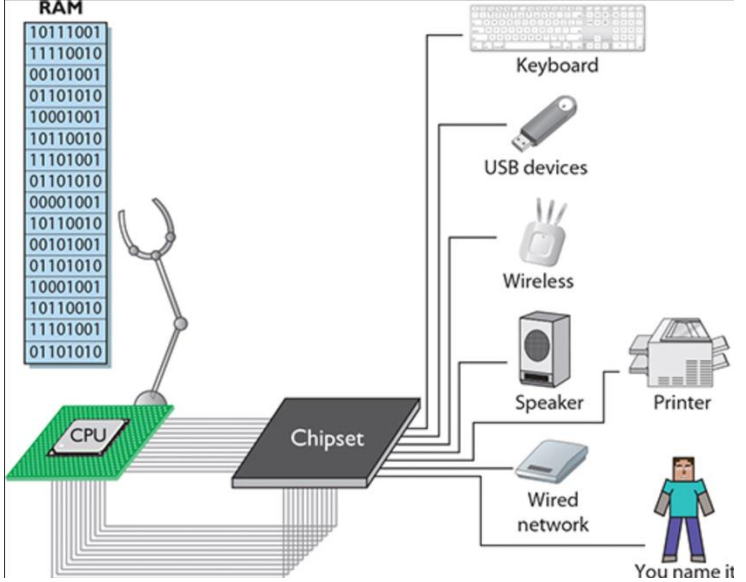
High/Low Voltages

When we talk ”logically” about 0s and 1s:

- Throughout the computer, it’s always high/low voltages.

Data and command to CPU?

- We need circuitry that converts CPU’s 0s and 1s to and from the outside world;
- This is accomplished by the controllers.



3-Digit Keyboard Entry Operation

CLEAR:

- Clears X & Y, preset Z FF: Z = J_X = 1;
- Clears all Q outputs.

1st key stroke:

- 74147 outputs BCD code (if 0 is pressed, Qs remain 0);
- OR gate output – T goes 1;
- Q goes 1 (and ignores T input for 20 ms);
- XYZ goes 100, activating Q8-Q11;
- BCD code of 1st stroke into 4-bit register Q8-Q11 as MSB.

2nd key stroke:

- XYZ goes 010, activating Q4-Q7;
- BCD of key stroke into Q4-Q7.

3rd key stroke: corresponding number is encoded in BCD and stored in Q0-Q3 as LSB.

- The 3-digit number stored can be sent to 7-Segment decoder/driver for display, or to Full Adder for computation.